

Deep Learning

Network of Perceptrons

Perceptron Convergence

Training set $\{(x_i, y_i)\}_{i=1}^N$, with $y_i \in \{-1, +1\}$.

- Perceptron update (on a mistake):

$$w_{t+1} = w_t + y_i x_i$$

- Assumption:

- **Bounded inputs:** $\|x_i\| \leq R$ for all i .

- **Linear separability with margin with a unit vector u :**

$$y_i(u^\top x_i) \geq \gamma \quad \gamma > 0. \quad \|u\| = 1.$$

- Let M be the number of mistakes/updates

Does perceptron learning rule make w_t closer to u ?

Can we find a bound on M ?

Perceptron Convergence

Progress Toward separator

- When the perceptron makes a mistake on (x_i, y_i) , it updates

$$w_{t+1} = w_t + y_i x_i$$

- Consider the dot product $w_t \cdot u$

$$w_{t+1} \cdot u = (w_t + y_i x_i) \cdot u = w_t \cdot u + y_i (x_i \cdot u)$$

Perceptron Convergence

Progress Toward separator

- When the perceptron makes a mistake on (x_i, y_i) , it updates

$$w_{t+1} = w_t + y_i x_i$$

- Consider the dot product $w_t \cdot u$

$$w_{t+1} \cdot u = (w_t + y_i x_i) \cdot u = w_t \cdot u + y_i (x_i \cdot u)$$

- By the margin assumption, $y_i (x_i \cdot u) \geq \gamma$

- After M mistakes

$$w_{t+1} \cdot u \geq w_t \cdot u + \gamma$$

$$w_M \cdot u \geq M\gamma$$

Perceptron keeps making **forward progress** toward a true separator.

Perceptron Convergence

Weights cannot grow too fast

Consider the squared norm

$$\|w_{t+1}\|^2 = \|w_t + y_i x_i\|^2 = \|w_t\|^2 + 2y_i(w_t \cdot x_i) + \|x_i\|^2$$

$$\|w_{t+1}\|^2 \leq \|w_t\|^2 + \|x_i\|^2 \leq \|w_t\|^2 + R^2$$

Perceptron Convergence

Weights cannot grow too fast

Consider the squared norm

$$\|w_{t+1}\|^2 = \|w_t + y_i x_i\|^2 = \|w_t\|^2 + 2y_i(w_t \cdot x_i) + \|x_i\|^2$$

$$\|w_{t+1}\|^2 \leq \|w_t\|^2 + \|x_i\|^2 \leq \|w_t\|^2 + R^2$$

After M mistakes $\|w_M\|^2 \leq MR^2 \Rightarrow \|w_M\| \leq R\sqrt{M}$

Weight magnitude grows at most **sublinearly** ($\sqrt{M}$)

Perceptron Convergence

- Can we have a bound on M (number of mistakes)
- We have $w_M \cdot u \geq M\gamma$ $\|w_M\| \leq R\sqrt{M}$
- Use Cauchy schwarz inequality

$$w_M \cdot u \leq \|w_M\| \|u\| = \|w_M\|$$

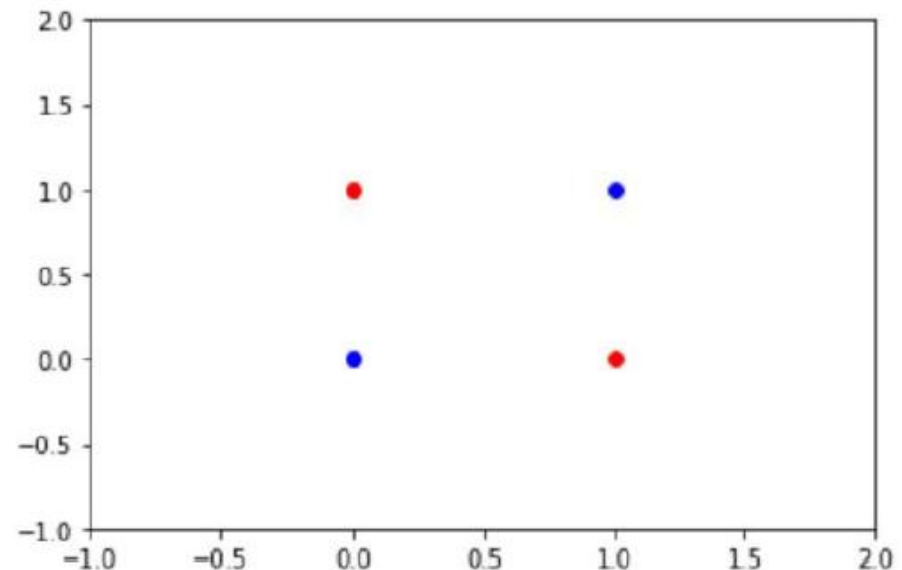
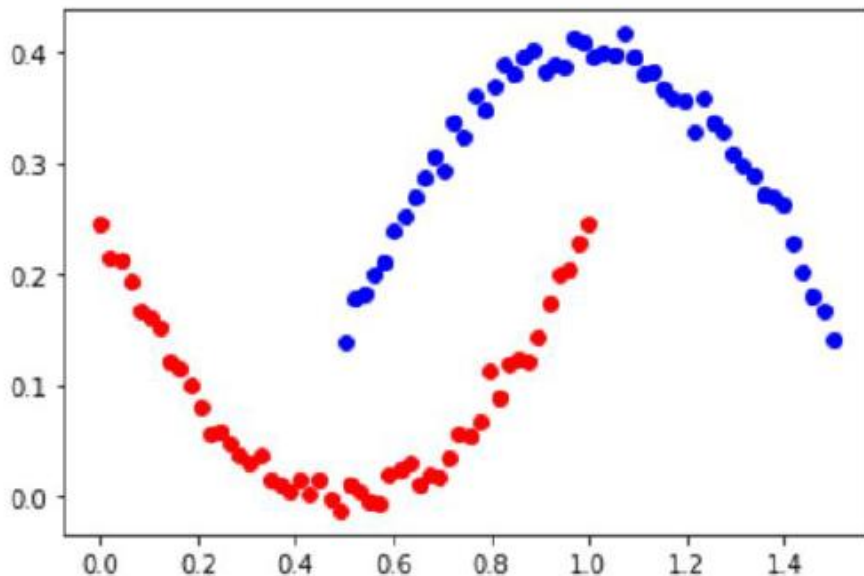
$$M\gamma \leq \|w_M\| \leq R\sqrt{M}$$

$$\sqrt{M}\gamma \leq R \Rightarrow \sqrt{M} \leq \frac{R}{\gamma} \Rightarrow M \leq \left(\frac{R}{\gamma}\right)^2$$

The number of mistakes/updates M is **finite** - bounded by $\left(R/\gamma\right)^2$

Linear Classifiers: Shortcomings

- The perceptron could perform tasks that many had considered impossible for a machine to do, such as [recognize images](#)
- In the late 1960s, it was becoming apparent that the perceptron could only learn a very specific format of problems: ones that exhibited a mathematical property called [linear separability](#).
- Lower capacity: data has to be linearly separable
- Some times no hyper-plane can separate the data (e.g. XOR data)
- <https://phiresky.github.io/neural-network-demo/>



Perceptrons and XOR

- Consider the XOR function
- Clearly, a single perception cannot represent the XOR function!

x1	x2	XOR	Condition
0	0	0	$w_0 < 0$
0	1	1	$w_2 + w_0 \geq 0$
1	0	1	$w_1 + w_0 \geq 0$
1	1	0	$w_1 + w_2 + w_0 < 0$

Chocolate

Tuna

Deliciousness

Perceptrons and XOR

- Consider the XOR function
- Clearly, a single perception cannot represent the XOR function!

x1	x2	XOR	Condition
0	0	0	$w_0 < 0$
0	1	1	$w_2 + w_0 \geq 0$
1	0	1	$w_1 + w_0 \geq 0$
1	1	0	$w_1 + w_2 + w_0 < 0$

$$w_0 < 0$$

$$w_2 + w_0 \geq 0 \quad \Rightarrow \quad w_2 \geq -w_0$$

$$w_1 + w_0 \geq 0 \quad \Rightarrow \quad w_1 \geq -w_0$$

$$w_1 + w_2 + w_0 < 0 \quad \Rightarrow \quad w_1 + w_2 < -w_0$$

Pre-processing

- In 1969, legendary AI scientist [Marvin Minsky](#), co-authored a highly critical book with [Seymour Papert](#) titled [Perceptrons: An Introduction to Computational Geometry](#). This book presented a rigorous and elegant proof of the perceptron's limitations
- Sometimes, data specific pre-processing makes the data linearly separable

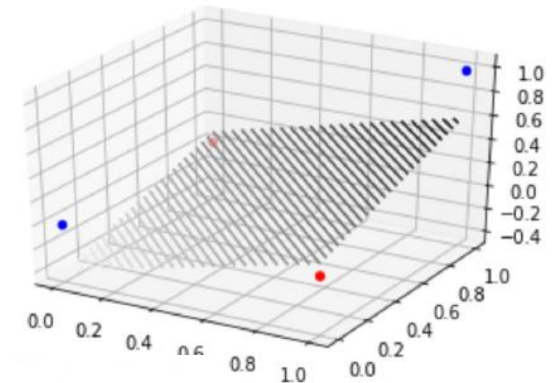
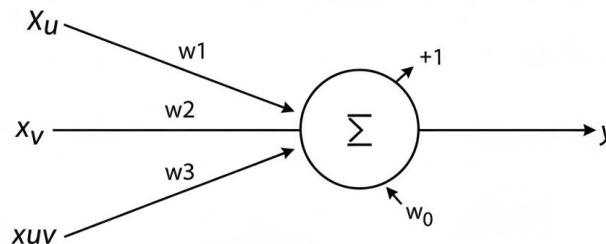
$$\phi(\mathbf{x}) = \phi(x_u, x_v) = (x_u, x_v, x_u x_v)$$

Chocolate and Tuna

- Consider the xor case

$$f(\mathbf{x}) = \sigma(\mathbf{w}^T \phi(\mathbf{x}) + b)$$

Consider the perceptron in the new space



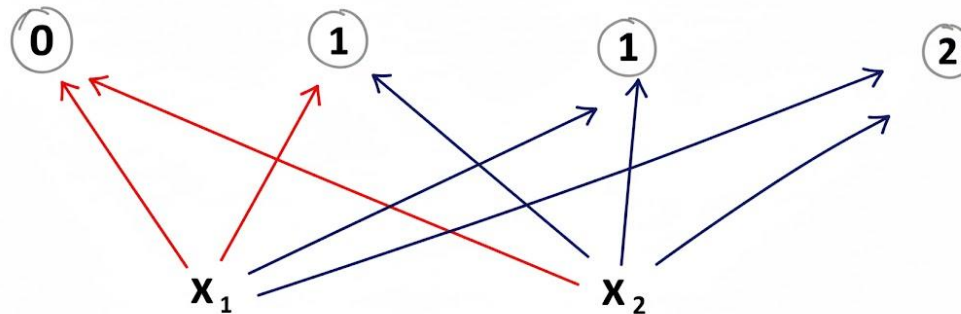
Pre-processing

- Feature design (or pre-processing) may a way to change the capacity without changing the bias but its hardwired by human.
- We want to have machines that could learn patterns/groupings on its own than through feature design (hardwired).
- In the Perceptron book, Minsky and Papert posited that, even though individual perceptrons had this limitation of linear separability, *combined stacks* or *layers* of perceptrons would not!

Multiple Perceptrons

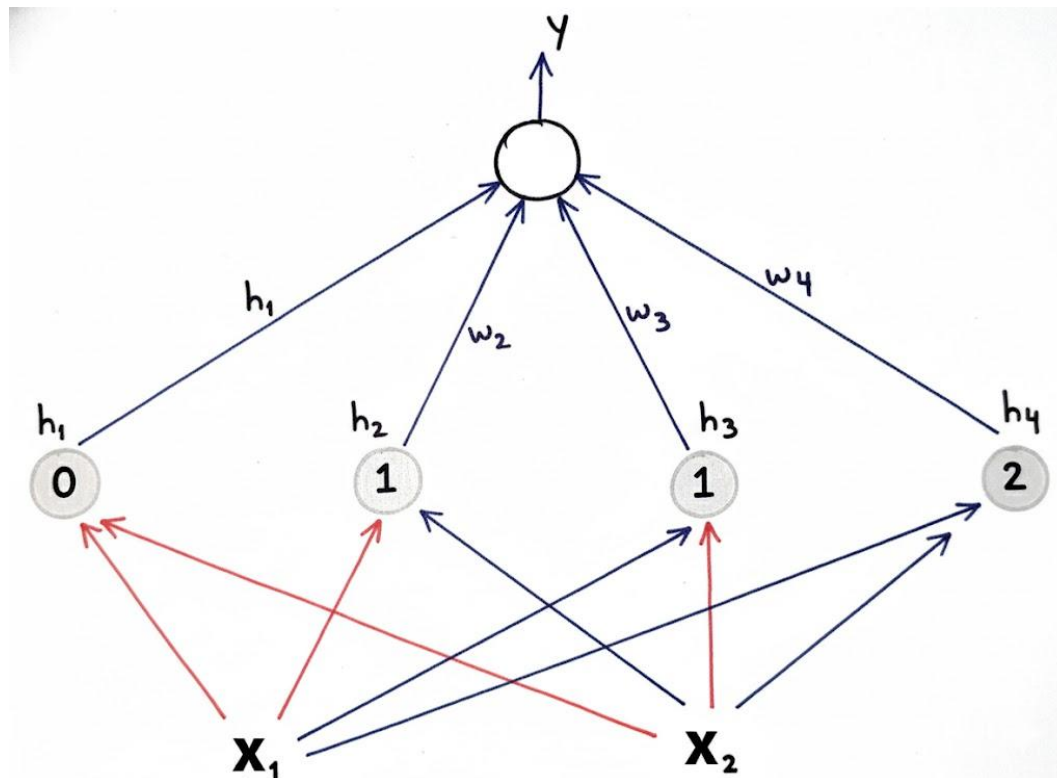
- Let's see if multiple perceptions can do this
- Consider 4 perceptions
- Notice, each of them fire for exactly one specific input pattern

$\rightarrow = -1$ and $\rightarrow = +1$



Multi-layer Perceptrons (MLP)

- Let's see if multiple perceptions can do this
- Connect the previous (hidden) ones and call it the output perceptron
- See if we can find a set of weights (w_i) to represent the XOR function
- hidden layer act as a set of "pattern detectors"



Multi-layer Perceptrons (MLP)

- Let's see if multiple perceptions can do this

Input x_1	Input x_2	h_1	h_2	h_3	h_4	Output y (Linear Combination)	XOR Result
0	0	1	0	0	0	$w_1 + w_0$	0
0	1	0	1	0	0	$w_2 + w_0$	1
1	0	0	0	1	0	$w_3 + w_0$	1
1	1	0	0	0	1	$w_4 + w_0$	0

Multi-layer Perceptrons (MLP)

- Let's see if multiple perceptions can do this
- Clearly possible to find such weights $\rightarrow$ represent the XOR function!

Input x_1	Input x_2	h_1	h_2	h_3	h_4	Output y (Linear Combination)	XOR Result
0	0	1	0	0	0	$w_1 + w_0$	0
0	1	0	1	0	0	$w_2 + w_0$	1
1	0	0	0	1	0	$w_3 + w_0$	1
1	1	0	0	0	1	$w_4 + w_0$	0

$$w_1 < -w_0 \quad w_2 \geq -w_0 \quad w_3 \geq -w_0 \quad w_4 < -w_0$$

Multi-layer Perceptrons (MLP)

- What about other 2-input Boolean functions?
- Possible to represent!
- While a single perceptron is limited, a **network** of perceptrons (specifically a Multilayer Perceptron or MLP) is a **universal approximator** for Boolean functions.
- **Any Boolean function** can be represented by a network with just **one hidden layer**
- Any Boolean function of n inputs can be exactly represented with 2^n perceptions in the hidden layer and 1 in the output layer!

Multi-layer Perceptrons (MLP)

- What about other 2-input Boolean functions?
- Possible to represent!
- While a single perceptron is limited, a **network** of perceptrons (specifically a Multilayer Perceptron or MLP) is a **universal approximator** for Boolean functions.
- **Any Boolean function** can be represented by a network with just **one hidden layer**
- Any Boolean function of n inputs can be exactly represented with 2^n perceptions in the hidden layer and 1 in the output layer!
 - There are exactly 2^n possible input patterns.
 - One can build a **2-layer network** with one hidden perceptron per input pattern.
 - Fire (output 1) if and only if the input equals one specific Boolean pattern
 - A single Boolean pattern is linearly separable from all others
 - You can choose weights and bias so that:

$$h_j(x) = \begin{cases} 1 & \text{if } x = \text{pattern } j \\ 0 & \text{otherwise} \end{cases}$$

Multi-layer Perceptrons (MLP)

- What about other 2-input Boolean functions?
- Possible to represent!
- While a single perceptron is limited, a **network** of perceptrons (specifically a Multilayer Perceptron or MLP) is a **universal approximator** for Boolean functions.
- **Any Boolean function** can be represented by a network with just **one hidden layer**
- Any Boolean function of n inputs can be exactly represented with 2^n perceptions in the hidden layer and 1 in the output layer!
- **Caveat:** the size of the hidden layer grows exponentially!

Moving on from Boolean functions

$$y = f(x), \text{ where } x \in \mathcal{R}^n \text{ and } y \in \mathcal{R}$$

- Can MLPs represent such functions?
- Perceptron's o/p is discontinuous!

$$\sigma(x) = \begin{cases} 1 & \text{when } x \geq 0 \\ -1 & \text{else} \end{cases}$$



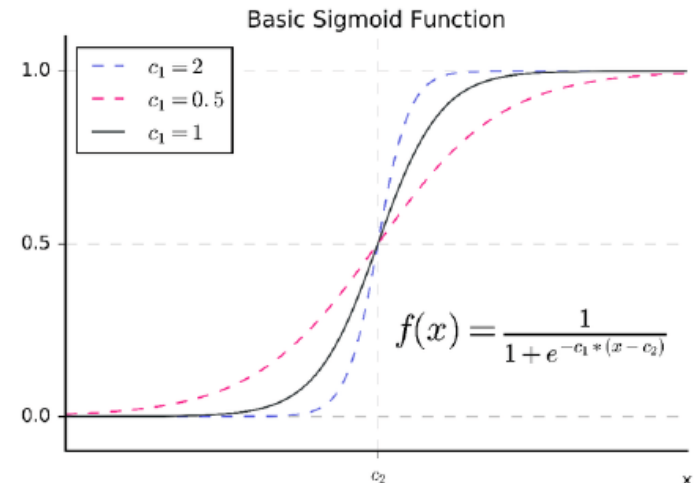
Sigmoid Neuron

- Many real world problems have non-binary outputs
- Perceptron only gives two outputs!
- Sigmoid neuron
- unlike a perceptron, the output is **not binary**, but **continuous**

sigmoid neurons are a **soft, trainable generalization** of perceptrons.

$$f(\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

- Range: (0,1)
- Smooth and continuous
- Differentiable everywhere
- Monotonic increasing



Sigmoid Neuron

- Many real world problems have non-binary outputs
- Perceptron only gives two outputs!
- Sigmoid neuron
- unlike a perceptron, the output is **not binary**, but **continuous**

$$f(\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

Perceptrons failed because:

- No gradient
- No multi-layer training

Sigmoid neurons enabled:

- **Backpropagation**
- **Multi-layer networks**

Learning **nonlinear representations**

